Introduction to Artificial Intelligence (IAI)

Car Mechanic Expert System

A Rule-Based Decision Support Tool

Decide. Defend. Drive safe.

Project Report	IAI Project
Submitted to	Sir Wasiq Noor
Submitted by	M Ahmed Khan
Roll Number	CT-245
Date	09 November 2026
Live Link	https://carmechanicexpertsystem-u5bxxwfrhat8mvdljvpjme.streamlit.app/

1. Title

Car Mechanic Expert System: A Rule-Based Decision Support Tool

This project is called the **IAI Expert System: Car Mechanic Assistant**. It is a web application that helps a normal car owner understand how serious a car problem is. It is built with Python and Streamlit and it works in English and Urdu.

Live link of the project: https://carmechanicexpertsystem-u5bxxwfrhat8mvdljvpjme.streamlit.app/

2. Introduction

Car problems are confusing for most people. When a warning light turns on or the car makes a strange noise, the owner often does not know if the car is safe to drive, if it can wait until next week, or if it needs a tow truck right now. Because of this, people either panic and spend too much money, or they ignore a serious problem and put their life in danger.

A **decision support tool** helps in this situation. It asks simple questions, checks the answers against a set of rules, and gives a clear suggestion with reasons. This project does exactly that. The user answers a few Yes/No questions about the car problem, and the system calculates a risk score, decides an urgency level, tells the user if they can drive the car, explains why it made this decision, and gives a report that can be shown to a mechanic.

The system is a **rule-based expert system**. It does not replace a qualified mechanic and it never gives a final diagnosis. It only supports the owner in taking a safe and sensible decision.

3. Explanation of the Project

The project does both **decision making** (it decides the urgency level and whether the car is safe to drive) and **decision support** (it explains and defends its suggestion, gives an action plan and a report for the mechanic).

3.1 How the system works

- The user chooses one of **10 car problem categories**: engine will not start, overheating, brakes, strange noise or vibration, battery or electrical, tyres, oil leak or smoke, AC, steering, and gear or clutch.
- If the user does not know the problem, **Guided mode** asks 16 screening questions and finds the closest matching problem.
- A **decision tree** then asks Yes/No questions one by one. Each answer leads to the next question.
- Serious answers add **risk points**. Some questions also ask for a number, such as battery voltage, engine temperature, oil level, tyre pressure or kilometres since the last service.
- The system calculates a **risk score from 0 to 100** and decides the urgency level.
- The final screen shows the decision, the reasons, an action plan and a downloadable mechanic report.

3.2 Urgency levels

Risk score	Urgency level	What it means
0 - 19	SELF-CARE	Simple problem. The user can fix it.
20 - 39	ROUTINE	Book a service this week.
40 - 69	URGENT	See a mechanic today.
70 - 100	EMERGENCY	Stop the car and call a tow truck.

3.3 Main features

- Guided mode with 16 screening questions for users who do not know their problem.
- 10 problem categories with 6 questions each, plus report-value inputs.
- Risk scoring with emergency minimums, so one dangerous sign can never give a low score.
- "Can I drive?" decision, "Why this decision?" and "Defending this suggestion" sections.
- Bilingual screen: English and Urdu (one button switches the language, Urdu uses right-to-left layout).
- Light and dark theme, and a layout that works on mobile phones.
- Downloadable mechanic report (.txt file).

3.4 Project files

File	What it does
car_ai.py	The Streamlit page: page setup, CSS design (light and dark theme), header, disclaimer, language and theme buttons.
codebase.py	The expert system brain: language picker T(), reset_system(), and run() with guided mode, the full if / elif decision tree, risk scoring and the final report.

Only two libraries are used: **streamlit** for the web screen and **datetime** for the report date. The rest is plain basic Python.

4. Code Explanation

Below are the main parts of the code, taken from the project files, with a short easy explanation of each.

4.1 Session state and reset

Streamlit runs the whole script again on every click. So `st.session_state` is used to remember the data (current step, risk score and reasons). `reset_system()` puts everything back to the start screen.

codebase.py - reset_system()

```
def reset_system():
    # Start a new consultation (the language choice is kept)
    s = st.session_state
    s.step = 'start'; s.patient_data = {}; s.report_data = {}
    s.risk = 0; s.reasons = []; s.balloons_done = False; s.area = "-|-"
    s.triage = {}; s.tri_match = ''; s.notice = ''
```

4.2 Language system (English / Urdu)

The function `T(en, ur)` returns the English or Urdu text depending on the chosen language. Every text on the screen is written with this function, so one button can change the whole app.

codebase.py - T()

```
def T(en, ur):
    # Pick UI text for the active language (English or Urdu)
    return en if st.session_state.lang == 'en' else ur
```

4.3 The decision tree and one question

The app is one big `if / elif` chain on `s.step`. Each step value is one screen. A "Yes" answer adds risk points and saves a reason, then moves to the next question. The reason is stored as "English|Urdu|points" so it can be shown in any language.

codebase.py - question 1 of 'Engine will not start'

```
elif step == 'ns1':
    # (question card with the progress bar is drawn here using st.markdown)
    c1, c2 = st.columns(2)
    if c1.button(YES, key="ns1y", type="primary"):
        s.patient_data['dash_lights_off'] = "Yes"
        s.risk = s.risk + 15
        s.reasons.append("Dashboard lights completely off: +15 risk|...|15")
        s.step = 'ns2'; st.rerun()
    if c2.button(NO, key="ns1n"):
        s.patient_data['dash_lights_off'] = "No"
        s.step = 'ns2'; st.rerun()
```

4.4 Emergency sign (safety minimum)

For very dangerous signs the code uses `max(s.risk + points, 70)`. This means the score can never stay low when a dangerous sign is reported. Here, a soft brake pedal immediately gives an EMERGENCY result.

codebase.py - brake problem, question 1

```
elif step == 'br1':
    c1, c2 = st.columns(2)
    if c1.button(YES, key="br1y", type="primary"):
        s.patient_data['soft_pedal'] = "Yes"
        s.risk = max(s.risk + 50, 70)
        s.reasons.append("Soft brake pedal: +50 risk (EMERGENCY sign)|...|50")
        s.step = 'end'; st.rerun()
    if c2.button(NO, key="br1n"):
        s.patient_data['soft_pedal'] = "No"; s.step = 'br2'; st.rerun()
```

4.5 Report-value inputs

Some steps ask for a number. `st.number_input` collects the value and the code adds risk points depending on the range. The example below is the battery voltage step.

codebase.py - battery voltage step

```
elif step == 'ns_bat':
    val = st.number_input(T('Battery voltage (V)', '...'), min_value=0.0,
                          max_value=20.0, value=12.6, step=0.1, format="%.1f",
                          key="ns_bat_in")
    if st.button(SUBMIT, key="ns_bat_b", type="primary"):
        s.report_data['battery_voltage'] = val; s.step = 'ns6'
        if val < 11.0:
            s.risk = s.risk + 30   # reason: battery voltage very low
        elif val < 12.2:
            s.risk = s.risk + 15   # reason: battery voltage low
        elif val > 15.0:
            s.risk = s.risk + 20   # reason: charging system needs a check
        st.rerun()
```

4.6 Guided mode

For users who do not know their problem, 16 screening questions are asked. Every "Yes" adds votes to the problem categories it could mean (saved in `s.triage`). At the end, the category with the most votes is chosen.

codebase.py - screening question 5 and the winner selection

```
elif step == 'tri5':
    # "The temperature gauge is in the red, or steam comes from under the bonnet."
    if c1.button(YES, key="tri5y", type="primary"):
        s.triage['oh'] = s.triage.get('oh', 0) + 4
        s.step = 'tri6'; st.rerun()
    if c2.button(NO, key="tri5n"): s.step = 'tri6'; st.rerun()

elif step == 'tri_end':
    # pick the problem code with the most votes (first one wins a tie)
    best = ''; best_v = 0
    for code in ['ns','oh','br','nv','el','ty','ol','ac','sr','gc']:
        v = s.triage.get(code, 0)
        if v > best_v: best_v = v; best = code
```

4.7 Final decision

On the final screen, the risk score is turned into an urgency level with simple `if / elif` rules. The same step then shows the suggestion, the "Can I drive?" decision, the reasons, the defence of the decision and the action plan.

codebase.py - step 'end' (urgency levels)

```
elif step == 'end':
    if s.risk > 100: s.risk = 100
    if s.risk >= 70:
        lvl = "emergency"; urg_en = "EMERGENCY"; lo_t = 70; hi_t = 100
        st.error(' EMERGENCY: Stop the car safely and call a tow truck!')
    elif s.risk >= 40:
        lvl = "urgent"; urg_en = "URGENT"; lo_t = 40; hi_t = 70
        st.warning(' URGENT: See a mechanic today')
    elif s.risk >= 20:
        lvl = "routine"; urg_en = "ROUTINE"; lo_t = 20; hi_t = 40
        st.info(' ROUTINE: Book a service this week')
    else:
        lvl = "selfcare"; urg_en = "SELF-CARE"; lo_t = 0; hi_t = 20
        st.success(' SELF-CARE: Simple fix you can do yourself')
```

4.8 Explainability and mechanic report

The system shows every stored reason with its points, finds the strongest factor, and explains why the level is not higher or lower. The report is a plain text string that the user can download and show to the mechanic.

codebase.py - strongest factor and report download

```
for r in s.reasons:
    parts = r.split("|"); pts = int(parts[2]); sum_pts = sum_pts + pts
    if pts > top_pts: top_pts = pts; top_en = parts[0]; top_ur = parts[1]
    rt = parts[1] if L == 'ur' else parts[0]
    st.write("• " + rt); hr = hr + "- " + rt + "\n"

hr = (T('CAR CHECK REPORT (decision support - NOT a diagnosis)', '...')) + "\n"
hr = hr + (T('Date: ', '...')) + datetime.now().strftime("%Y-%m-%d %H:%M") + "\n"
st.download_button(T('Download Report', '...'), data=hr,
                   file_name="mechanic_report.txt", mime="text/plain", key="dl")
```

4.9 Design ideas used

- **Decision tree** instead of a flat questionnaire: each answer leads to the next question.
- **Weighted scoring**: different symptoms add different risk points.
- **Safety minimum**: one dangerous sign can never give a low score.
- **Explainability**: the app shows its reasons and defends its own suggestion.
- **Bilingual storage**: texts are stored once as "English|Urdu|points".

5. Market Use Case

Most everyday drivers do not know car mechanics. When a problem appears, they do not know how serious it is, and a workshop visit can cost time and money. This tool checks the symptoms before the workshop visit, reduces panic and helps the owner take a safe decision.

It also helps mechanics. The downloadable report gives a clear and structured summary of the problem, so the mechanic can start the check faster.

Possible users:

- **Car owners:** to understand if the car is safe to drive.
- **Driving schools:** to teach students about warning signs.
- **Fleet managers:** to quickly check the condition of company vehicles.
- **Roadside help services:** to judge how urgent a call is before sending help.

Because the app supports English and Urdu and works on mobile phones, it can be used by many more people in Pakistan.

6. Limitations and Future Work

6.1 Limitations

- The system is **rule-based**. It follows fixed rules written by the programmer and it does not learn from data like AI or machine learning.
- It **does not replace a mechanic**. It gives decision support only and never a final diagnosis.
- It covers only **10 problem categories**, so rare or mixed problems may not be matched well.
- The risk points are fixed numbers. They are sensible, but they are not based on real workshop data.
- The user must enter some values (such as battery voltage or tyre pressure) by hand, and wrong inputs give a wrong result.
- Only two languages are supported: English and Urdu.

6.2 Future Work

- Build a **real diagnosis knowledge base** with more car problems and more detailed rules.
- Add **voice input**, so a driver can describe the problem by speaking.
- Support **more languages**.
- Make a **mobile app** for Android and iPhone.
- Add a machine learning part that learns from real repair records to improve the risk scores.
- Save the consultation history and let the user share the report directly with a mechanic.

7. Screenshots of the Project

Live link: https://carmechanicexpertsystem-u5bxxwfrhat8mvdljvpjme.streamlit.app/

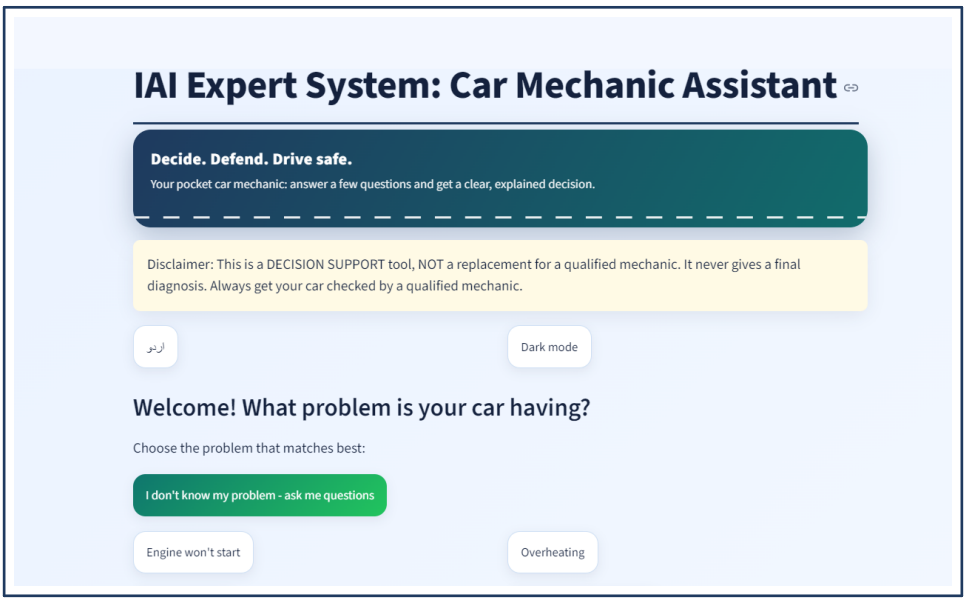


Figure 1: Welcome screen in English with the 10 problem categories and the guided mode button.



Figure 2: Welcome screen in Urdu (right-to-left layout) after pressing the language button.

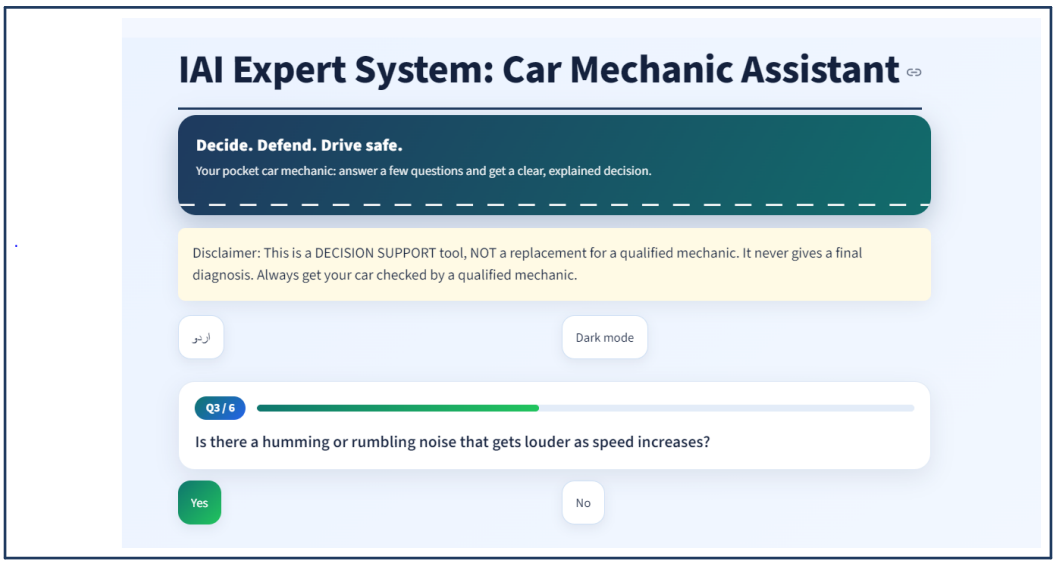


Figure 3: A Yes/No question card with the question number and the progress bar.

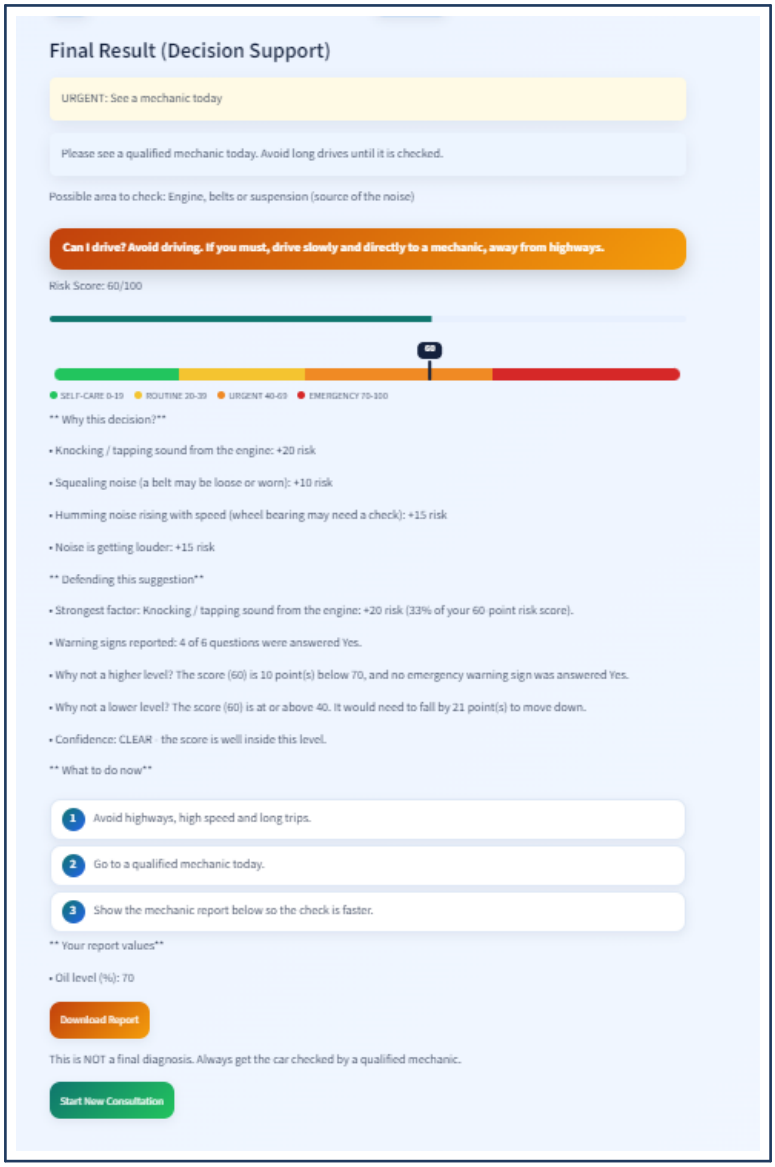


Figure 4: Final result with risk score, "Can I drive?" decision, reasons, action plan and the report download button.